ФГБОУ ВО «ЮГОРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

ИНЖЕНЕРНАЯ ШКОЛА ЦИФРОВЫХ ТЕХНОЛОГИЙ

МИНИ-ОТЧЁТ

по разработке серверной части веб-приложения

«Платформа онлайн-хакатонов и соревнований»

Выполнили: студенты группы ПИ41Б, команда № 3

Полуянов, Бойдаченко, Абдурахман, Борисов

Проверил: Исмагулов Милан Ерикович

г. Ханты-Мансийск, 2026 г.

1 ЦЕЛЬ И РЕЗУЛЬТАТ РАБОТЫ

Цель работы — реализовать работоспособный бэкенд платформы онлайн-хакатонов с постоянным хранением, контролем доступа, серверным дедлайном сдачи и API для команд, организаторов и экспертов.

Разработаны авторизация и роли, управление хакатонами и кейсами, команды и приглашения, приём ссылок и архивов, экспертная оценка по критериям, публичный рейтинг и журнал уведомлений. API доступен через Swagger UI.

2 ССЫЛКА НА АКТУАЛЬНЫЙ КОД

Код серверной части опубликован в открытом репозитории GitHub в ветке backend. Адрес ветки:
https://github.com/Andrey787878/hackhaton/tree/backend

В ветке размещены исходный код, миграции БД, Docker Compose, тесты и инструкция запуска. Демо-пароли и локальная конфигурация исключены из Git.

3 ВЫБРАННЫЙ СТЕК ТЕХНОЛОГИЙ

Python 3.11 и FastAPI выбраны для типизированного HTTP API, автоматической валидации входных данных и генерации OpenAPI/Swagger. Pydantic описывает схемы запросов и ответов.

PostgreSQL 17 хранит пользователей, права, хакатоны, команды, метаданные сдач и оценки. SQLAlchemy 2 отделяет модели данных от маршрутов, Alembic обеспечивает версионируемые миграции.

MinIO предоставляет S3-совместимое приватное хранение архивов; в SQL находятся только метаданные и ключ объекта. Docker Compose запускает API, БД, MinIO и служебные контейнеры миграции и инициализации bucket. PyJWT и bcrypt применяются для сессий и хэширования паролей; Pytest проверяет бизнес-сценарии с реальными PostgreSQL и MinIO.

4 АРХИТЕКТУРА И СТРУКТУРА

HTTP-запрос проходит через маршрут-контроллер, проверку пользователя и входных данных, сервис бизнес-логики, ORM и PostgreSQL. Архив передаётся через S3-адаптер в MinIO. Основные каталоги представлены ниже.

```
hackhaton/
├── app/
│   ├── main.py, config.py, db.py
│   ├── models.py, dependencies.py
│   ├── auth_routes.py, auth_service.py
│   ├── rbac_routes.py, rbac_service.py
│   ├── hackathon_routes.py, hackathon_service.py
│   ├── team_routes.py, team_service.py
│   ├── submission_routes.py, submission_service.py
│   ├── scoring_routes.py, scoring_service.py
│   ├── notification_routes.py, notification_service.py
│   ├── *_schemas.py, storage.py, storage_bootstrap.py
├── alembic/versions/      миграции PostgreSQL
├── tests/                 интеграционные тесты
├── scripts/               настройка и демосценарий
├── Dockerfile, Dockerfile.minio
├── docker-compose.yml, compose.test.yml
└── requirements.txt       зависимости Python
```

Файлы *_routes.py выполняют роль контроллеров: принимают HTTP-запросы и возвращают ответы. Файлы *_service.py содержат правила доступа, дедлайны и транзакции. В models.py описаны ORM-модели, в *_schemas.py — контракты API. Каталог alembic/versions хранит историю схемы БД, tests/ — автоматические проверки.

5 ПОДТВЕРЖДЕНИЕ РАБОТОСПОСОБНОСТИ

Swagger UI запущенного сервера содержит 73 операции в восьми разделах. На рисунках 1–4 последовательно показаны все группы API. Полная схема OpenAPI доступна по адресу /openapi.json.

На рисунке 1 показаны авторизация и начало управления ролями и правами.

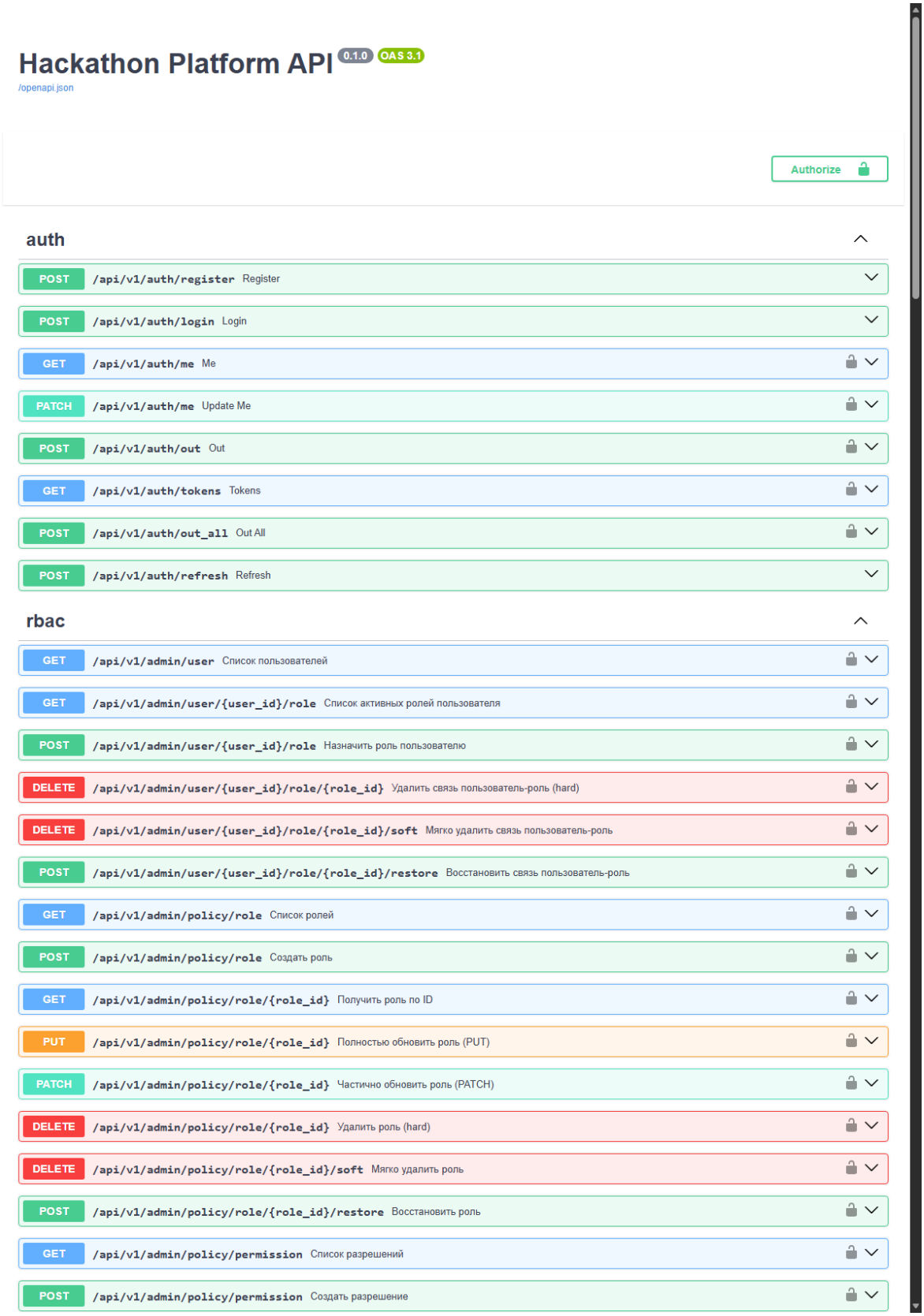


Рисунок 1 – Swagger UI: авторизация и управление доступом, часть 1

На рисунке 2 показаны оставшиеся маршруты прав доступа и маршруты хакатонов.



Рисунок 2 – Swagger UI: управление доступом, часть 2, и хакатоны

На рисунке 3 показаны команды, сдача решений и экспертная оценка.

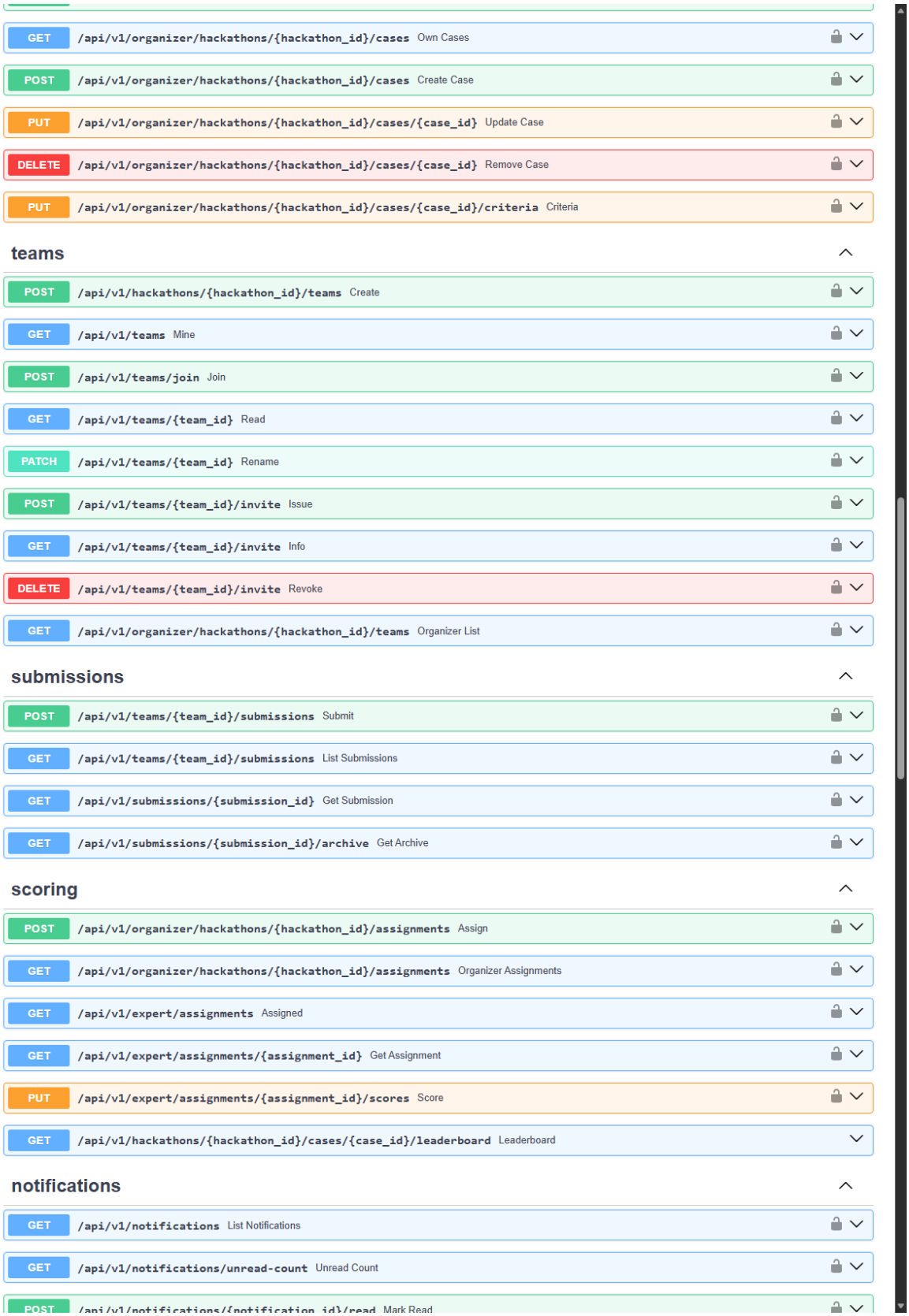


Рисунок 3 – Swagger UI: команды, решения и оценка

На рисунке 4 показаны уведомления и проверка работоспособности.

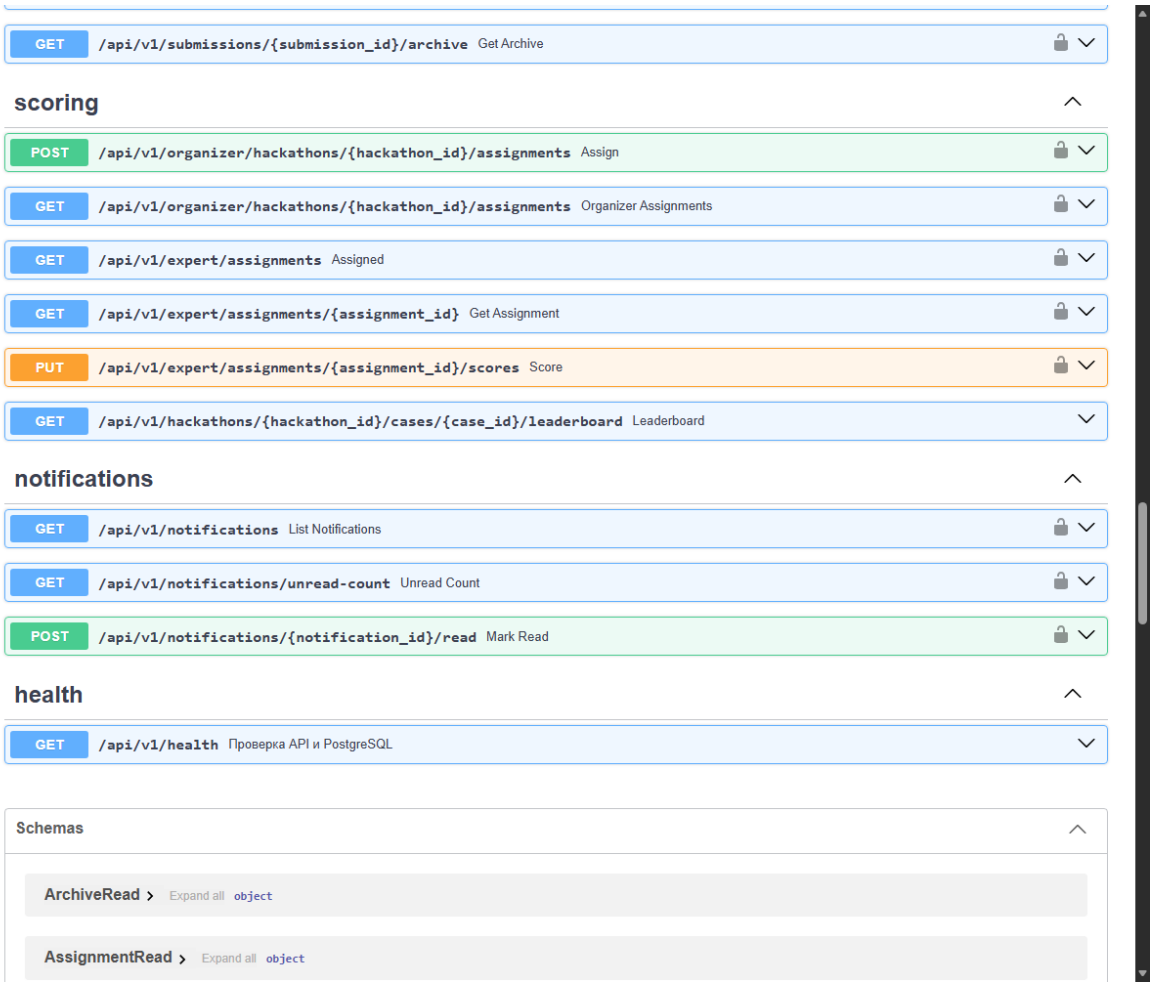


Рисунок 4 – Swagger UI: уведомления и проверка API

На рисунке 5 приведён успешный ответ GET /api/v1/health: сервер и подключение к PostgreSQL исправны.

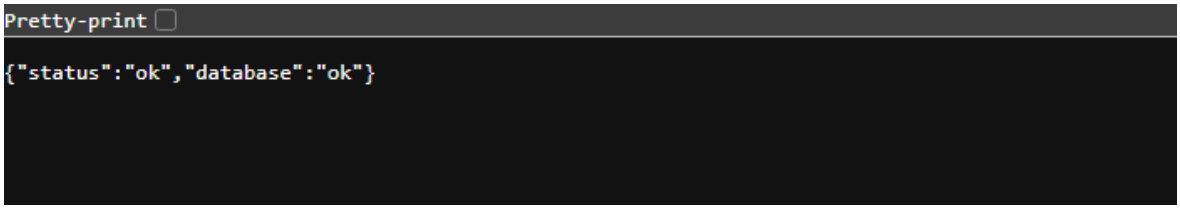


Рисунок 5 – Ответ проверки API и базы данных

На рисунке 6 представлен ответ публичного GET-запроса рейтинга для кейса № 3: команда получила итог 85,00 балла по весовым критериям.

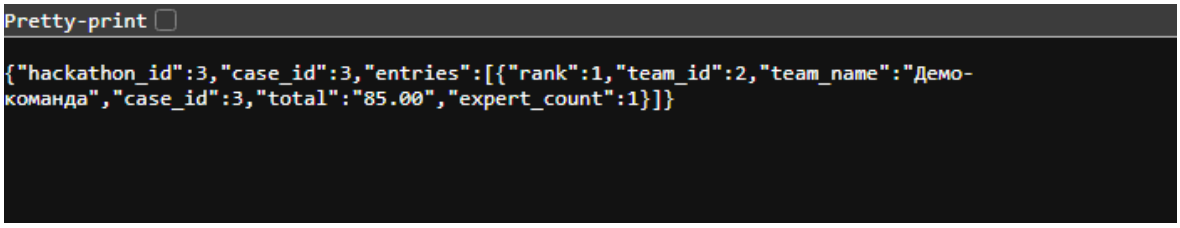


Рисунок 6 – Публичный рейтинг по результатам экспертной оценки

Дополнительно выполнены 87 интеграционных тестов. Сквозной сценарий проверил публикацию хакатона, создание команды, сдачу архива, назначение эксперта, оценку и рейтинг. Сдача после дедлайна отклонена с HTTP 409; анонимное скачивание архива — с HTTP 401. Проверка alembic check не обнаружила расхождений между моделями и миграциями.

6 ЗАКЛЮЧЕНИЕ

Основная серверная бизнес-логика платформы реализована и доступна через версионированный API /api/v1. Запуск воспроизводится по README с помощью Docker Compose. Подтверждены работа API, сохранение данных и архивов, разграничение прав, соблюдение дедлайна и расчёт рейтинга.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Задание 3. Платформа для онлайн-хакатонов и соревнований. URL: https://yusu-source.github.io/GH_pages/03_hackathon.html (дата обращения: 28.09.2026).
2. ГОСТ 7.32—2017. Отчёт о научно-исследовательской работе. Структура и правила оформления.
3. Репозиторий проекта: https://github.com/Andrey787878/hackhaton.